Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 5
з дисципліни: «Теорія алгоритмів»
Тема: «Алгоритми на графах. Мінімальне кістякове дерево»

Варіант № 22

Виконав:
Студент групи АІ-254
Церковний О. Г.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів Прима (Prim) і Крускала (Kruskal) для побудови мінімального кістякового дерева.

Завдання:

1. Побудувати матриці та списки суміжності для зважених неорієнтованих графів за варіантами завдань.

2. Побудувати за допомогою алгоритму Прима мінімальне кістякове дерево (МКД) для графа за варіантами завдань та показати графічно хід побудови МКД за алгоритмом Пріма (Таблиця 5.1).

3. Запрограмувати алгоритм Пріма. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови МКД співпадають.

4. Побудувати за допомогою алгоритму Крускала МКД для графа за варіантами завдань та показати графічно хід побудови МКД за алгоритмом Крускала (Таблиця 5.2).

5. Запрограмувати алгоритм Крускала. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови МКД співпадають.

6. Порівняти результати побудови МКД за двома алгоритмами, пояснити різницю, якщо вона буде.

Варіант № 22. Вхідні дані – зважений неорієнтований граф із 8 вершинами та 12 ребрами, наведений на рисунку 5.1.

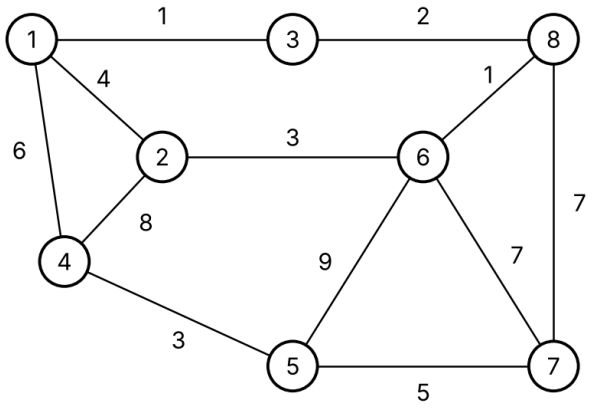


Рисунок 5.1 – Граф за варіантом № 22

V = {1, 2, 3, 4, 5, 6, 7, 8}, |V| = 8.

E = {(1,3)=1, (3,8)=2, (1,2)=4, (1,4)=6, (2,6)=3, (2,4)=8, (6,8)=1, (5,6)=9, (6,7)=7, (7,8)=7, (4,5)=3, (5,7)=5}, |E| = 12.

Примітка: у графі варіанта деякі ваги ребер дублюються (1-3 і 6-8 мають вагу 1; 2-6 і 4-5 – вагу 3; 6-7 і 7-8 – вагу 7). Згідно з приміткою до варіантів ваги залишено без змін: при рівних вагах вибирається ребро, яке знайдено першим (для Пріма – при переборі вершин за зростанням номера, для Крускала – ребро з меншими номерами вершин). На загальну вагу МКД це не впливає.

Результати виконання завдання:

1. Матриця та список суміжності графа

Матрицю суміжності (вагових коефіцієнтів) графа варіанта 22 наведено в таблиці 5.1. На перетині рядка i та стовпця j записано вагу ребра (i, j), якщо ребро існує, та «0» – якщо ребра немає. Матриця симетрична, оскільки граф неорієнтований; її розмір $|V|^2 = 64$.

Таблиця 5.1 – Матриця суміжності (вагових коефіцієнтів) графа

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
1	0	**4**	**1**	**6**	0	0	0	0
2	**4**	0	0	**8**	0	**3**	0	0
3	**1**	0	0	0	0	0	0	**2**
4	**6**	**8**	0	0	**3**	0	0	0
5	0	0	0	**3**	0	**9**	**5**	0
6	0	**3**	0	0	**9**	0	**7**	**1**
7	0	0	0	0	**5**	**7**	0	**7**
8	0	0	**2**	0	0	**1**	**7**	0

Список суміжності графа показано на рисунку 5.2. Масив містить голови списків суміжності для кожної вершини; кожен вузол списку зберігає номер суміжної вершини, вагу ребра та покажчик на наступний вузол. Сусідні вершини розташовані у числовому порядку.

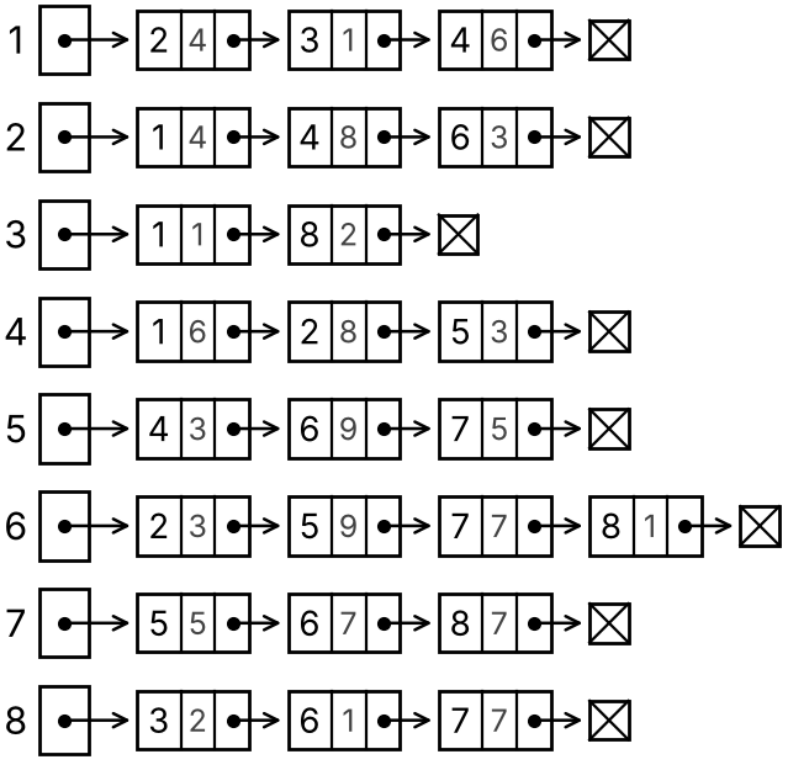


Рисунок 5.2 – Список суміжності графа (у вузлі: вершина | вага | покажчик)

У текстовому вигляді (вершина: сусід(вага)):

```
1: 2(4) -> 3(1) -> 4(6) -> null
2: 1(4) -> 4(8) -> 6(3) -> null
3: 1(1) -> 8(2) -> null
4: 1(6) -> 2(8) -> 5(3) -> null
5: 4(3) -> 6(9) -> 7(5) -> null
6: 2(3) -> 5(9) -> 7(7) -> 8(1) -> null
7: 5(5) -> 6(7) -> 8(7) -> null
8: 3(2) -> 6(1) -> 7(7) -> null
```

2. Побудова МКД за алгоритмом Пріма

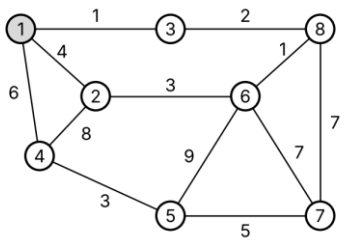
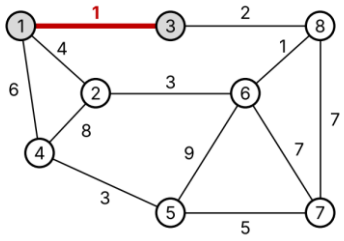
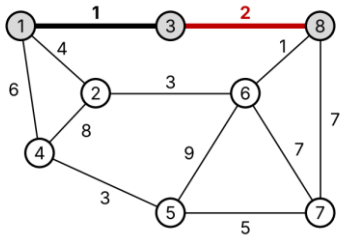
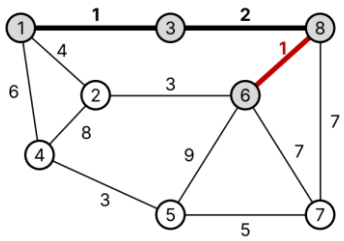
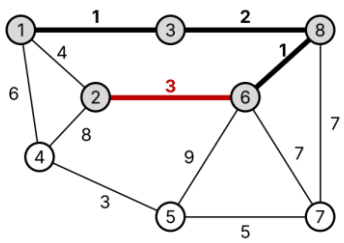
Алгоритм Пріма будує МКД як послідовність піддерев, що розширюються: на кожній ітерації до дерева жадібно додається найближча вершина, тобто вибирається ребро мінімальної ваги, яке з'єднує вершину дерева V_T з вершиною множини $V – V_T$. Псевдокод алгоритму наведено на лістингу 5.1.

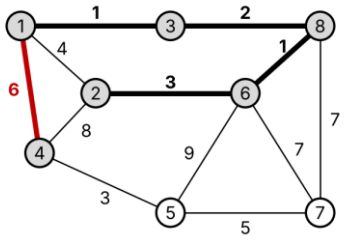
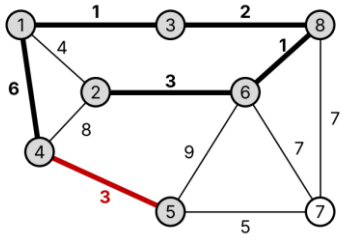
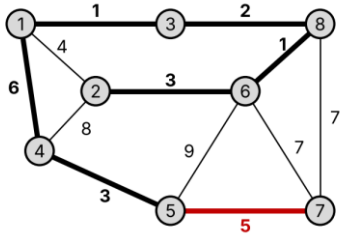
Лістинг 5.1 – Псевдокод алгоритму Пріма

```
Алгоритм Prim (G)
// Вхідні дані: зважений зв'язний граф G = (V, E)
// Вихідні дані: E_T, множина ребер, що складають MST графа G
V_T <- {v0}        // довільна початкова вершина (v0 = 1)
E_T <- ∅
for i <- 1 to |V| - 1 do
    знайти ребро мінімальної ваги e* = (v*, u*) серед усіх
    ребер (v, u), таких, що v ∈ V_T, u ∈ V - V_T
    V_T <- V_T ∪ {u*}
    E_T <- E_T ∪ {e*}
return E_T
```

Хід побудови МКД за алгоритмом Пріма (початкова вершина – 1) наведено в таблиці 5.2. Позначення: V_T – вершини МКД (на рисунку зафарбовані сірим); E – множина примежових ребер, що з'єднують V_T з $V – V_T$; $V – V_T$ – непереглянуті вершини; e* – ребро з мінімальною вагою, обране на поточному кроці (на рисунку виділено червоним); E_T – множина ребер МКД.

Таблиця 5.2 – Хід побудови МКД за алгоритмом Пріма

Зображення	V_T	E	$V - V_T$	e*	E_T
	{1}	(1,2) = 4 (1,3) = 1 (1,4) = 6	{2,3,4,5,6,7,8}	–	∅
	{1,3}	(1,2) = 4 (1,4) = 6 (3,8) = 2	{2,4,5,6,7,8}	(1,3) = 1	(1,3) = 1
	{1,3,8}	(1,2) = 4 (1,4) = 6 (8,6) = 1 (8,7) = 7	{2,4,5,6,7}	(3,8) = 2	(1,3) = 1 (3,8) = 2
	{1,3,8,6}	(1,2) = 4 (1,4) = 6 (8,7) = 7 (6,2) = 3 (6,5) = 9 (6,7) = 7	{2,4,5,7}	(8,6) = 1	(1,3) = 1 (3,8) = 2 (8,6) = 1
	{1,3,8,6,2}	(1,4) = 6 (8,7) = 7 (6,5) = 9 (6,7) = 7 (2,4) = 8	{4,5,7}	(6,2) = 3	(1,3) = 1 (3,8) = 2 (8,6) = 1 (6,2) = 3

Зображення	V_T	E	$V - V_T$	e*	E_T
	{1,3,8,6,2,4}	(8,7) = 7 (6,5) = 9 (6,7) = 7 (4,5) = 3	{5,7}	(1,4) = 6	(1,3) = 1 (3,8) = 2 (8,6) = 1 (6,2) = 3 (1,4) = 6
	{1,3,8,6,2,4,5}	(8,7) = 7 (6,7) = 7 (5,7) = 5	{7}	(4,5) = 3	(1,3) = 1 (3,8) = 2 (8,6) = 1 (6,2) = 3 (1,4) = 6 (4,5) = 3
	{1,3,8,6,2,4,5,7}	–	{ }	(5,7) = 5	(1,3) = 1 (3,8) = 2 (8,6) = 1 (6,2) = 3 (1,4) = 6 (4,5) = 3 (5,7) = 5

Пояснення до кроків. На кроці 1 з трьох ребер, інцидентних вершині 1, обрано найлегше (1,3)=1. Далі обрано (3,8)=2, потім (8,6)=1. На кроці 4 серед примежових ребер найменшу вагу має (6,2)=3. На кроці 5 ребро (1,2) вже не розглядається, бо обидві його вершини належать дереву (утворився б цикл), тому обрано (1,4)=6 – найлегше ребро, що веде до вершин 4, 5, 7. Після цього стають доступними (4,5)=3 та (5,7)=5, які завершують побудову.

Мінімальне кістякове дерево побудовано: E_T = {(1,3)=1, (3,8)=2, (8,6)=1, (6,2)=3, (1,4)=6, (4,5)=3, (5,7)=5}, кількість ребер |V| – 1 = 7.

Його вартість: $e^* = \sum e_i^* = 1 + 2 + 1 + 3 + 6 + 3 + 5 = 21$.

3. Програмна реалізація алгоритму Пріма

Програмну реалізацію алгоритму Пріма мовою C++ наведено на лістингу 5.2. Граф подано матрицею суміжності (таблиця 5.1); вершини 1–8 зберігаються з індексами 0–7, а при виведенні перетворюються на номери з варіанта. Побудова починається з вершини 1.

Лістинг 5.2 – Програмний код алгоритму Пріма

```
#include <iostream>
#include <cstring>
using namespace std;
#define INF 9999999
// кількість вершин графа (варіант 22)
#define V 8
// матриця суміжності (ваг); вершини 1..8 -> індекси 0..7
int G[V][V] = {
//   1    2    3    4    5    6    7    8
  {  0,   4,   1,   6,   0,   0,   0,   0}, // 1
  {  4,   0,   0,   8,   0,   3,   0,   0}, // 2
  {  1,   0,   0,   0,   0,   0,   0,   2}, // 3
  {  6,   8,   0,   0,   3,   0,   0,   0}, // 4
  {  0,   0,   0,   3,   0,   9,   5,   0}, // 5
  {  0,   3,   0,   0,   9,   0,   7,   1}, // 6
  {  0,   0,   0,   0,   5,   7,   0,   7}, // 7
  {  0,   0,   2,   0,   0,   1,   7,   0}  // 8
};
int main() {
    int no_edge = 0;          // кількість ребер у МКД
    bool selected[V];         // вершини, що вже включені до МКД (V_T)
    memset(selected, false, sizeof(selected));
    selected[0] = true;       // починаємо з вершини 1
    int x, y, total = 0;
    cout << "Edge : Weight" << endl;
    while (no_edge < V - 1) {
        int min = INF;
        x = 0; y = 0;
        // пошук ребра мінімальної ваги (v in V_T, u in V - V_T)
        for (int i = 0; i < V; i++) {
            if (selected[i]) {
                for (int j = 0; j < V; j++) {
```

```
                    if (!selected[j] && G[i][j]) {
                        if (min > G[i][j]) {
                            min = G[i][j];
                            x = i; y = j;
                        }
                    }
                }
            }
        }
        cout << x + 1 << " - " << y + 1 << " :  " << G[x][y] << endl;
        total += G[x][y];
        selected[y] = true;   // V_T <- V_T U {u*}
        no_edge++;
    }
    cout << "Total weight: " << total << endl;
    return 0;
}
```

Результат виконання програми наведено на лістингу 5.3.

Лістинг 5.3 – Результат виконання програми

```
Edge : Weight
1 - 3 :  1
3 - 8 :  2
8 - 6 :  1
6 - 2 :  3
1 - 4 :  6
4 - 5 :  3
5 - 7 :  5
Total weight: 21
```

Порівняння ручної та автоматизованої побудови наведено в таблиці 5.3.

Таблиця 5.3 – Порівняння ручної та програмної побудови МКД (алгоритм Пріма)

Крок	**Ручна побудова (таблиця 5.2)**	**Програма (лістинг 5.3)**
1	(1,3) = 1	1 - 3 : 1
2	(3,8) = 2	3 - 8 : 2

3	(8,6) = 1	8 - 6 : 1
4	(6,2) = 3	6 - 2 : 3
5	(1,4) = 6	1 - 4 : 6
6	(4,5) = 3	4 - 5 : 3
7	(5,7) = 5	5 - 7 : 5
Σ	21	Total weight: 21

Як видно з таблиці 5.3, послідовність ребер та загальна вага МКД (21), отримані програмою, повністю збігаються з результатами ручної побудови.

4. Побудова МКД за алгоритмом Крускала

Алгоритм Крускала спочатку сортує ребра графа в неспадному порядку їх ваг, а потім, починаючи з порожнього підграфа, переглядає відсортований список і додає чергове ребро до МКД, якщо воно не утворює цикл. Для перевірки на цикл використовується система неперетинних множин DSU (UNION-FIND): ребро (u, v) утворює цикл, якщо FIND(u) = FIND(v). Псевдокод наведено на лістингу 5.4.

Лістинг 5.4 – Псевдокод алгоритму Крускала

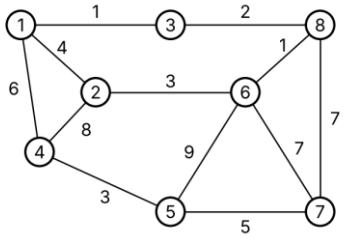
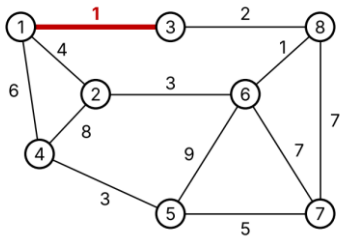
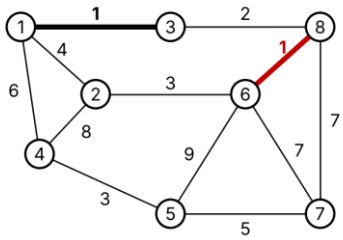
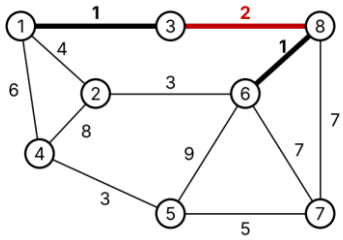
```
Алгоритм Kruskal (G)
// Вхідні дані: зважений зв'язний граф G = (V, E)
// Вихідні дані: E_T, множина ребер мінімального кістякового дерева
Сортування E в неспадному порядку ваг: e*_0 <= ... <= e*_(|E|-1)
E_T <- ∅
ecounter <- 0   // кількість ребер, доданих до E_T
k <- 0          // номер переглянутого ребра
while ecounter < |V| - 1 do
    if E_T ∪ {e*_k} – ациклічний граф   // FIND(u) != FIND(v)
        E_T <- E_T ∪ {e*_k}; ecounter <- ecounter + 1   // UNION(u, v)
    k <- k + 1
return E_T
```

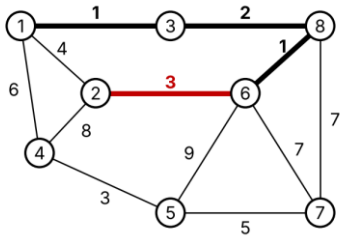
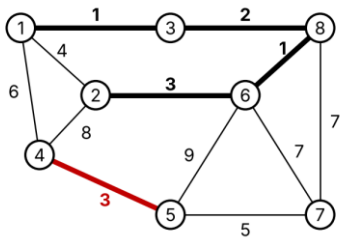
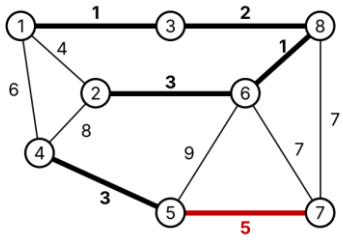
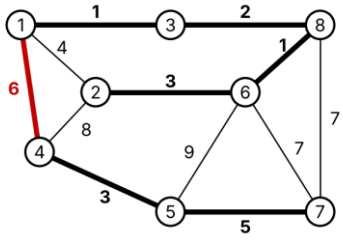
Відсортований у неспадному порядку ваг список ребер:

E = {(1,3)=1, (6,8)=1, (3,8)=2, (2,6)=3, (4,5)=3, (1,2)=4, (5,7)=5, (1,4)=6, (6,7)=7, (7,8)=7, (2,4)=8, (5,6)=9}.

Хід побудови МКД за алгоритмом Крускала наведено в таблиці 5.4. Позначення: e_k* – ребро, що розглядається на кроці k (на рисунку виділено червоним); DSU – стан неперетинних множин після кроку; E_T – множина ребер МКД.

Таблиця 5.4 – Хід побудови МКД за алгоритмом Крускала

Зображення	e_k* та перевірка	DSU	E_T
	Початковий стан. Ребра відсортовано.	{1} {2} {3} {4} {5} {6} {7} {8}	{ }
	k=0: (1,3) = 1 FIND(1)≠FIND(3) – циклу немає, ребро додано	{2} {1,3} {4} {5} {6} {7} {8}	(1,3) = 1
	k=1: (6,8) = 1 FIND(6)≠FIND(8) – циклу немає, ребро додано	{2} {1,3} {4} {5} {7} {6,8}	(1,3) = 1 (6,8) = 1
	k=2: (3,8) = 2 FIND(3)≠FIND(8) – циклу немає, ребро додано	{2} {4} {5} {7} {1,3,6,8}	(1,3) = 1 (6,8) = 1 (3,8) = 2

Зображення	e_k* та перевірка	DSU	E_T
	k=3: (2,6) = 3 FIND(2)≠FIND(6) – циклу немає, ребро додано	{4} {5} {7} {1,2,3,6,8}	(1,3) = 1 (6,8) = 1 (3,8) = 2 (2,6) = 3
	k=4: (4,5) = 3 FIND(4)≠FIND(5) – циклу немає, ребро додано	{4,5} {7} {1,2,3,6,8}	(1,3) = 1 (6,8) = 1 (3,8) = 2 (2,6) = 3 (4,5) = 3
	k=6: (5,7) = 5 FIND(5)≠FIND(7) – циклу немає, ребро додано	{4,5,7} {1,2,3,6,8}	(1,3) = 1 (6,8) = 1 (3,8) = 2 (2,6) = 3 (4,5) = 3 (5,7) = 5
	k=7: (1,4) = 6 FIND(1)≠FIND(4) – циклу немає, ребро додано |E_T| = |V| – 1 = 7. Дерево побудовано.	{1,2,3,4,5,6,7,8}	(1,3) = 1 (6,8) = 1 (3,8) = 2 (2,6) = 3 (4,5) = 3 (5,7) = 5 (1,4) = 6

Після ребра (4,5)=3 (k = 4) розглянуто ребро (1,2)=4 (k = 5). Його відхилено, оскільки FIND(1)=FIND(2), а додавання цього ребра утворило б цикл 1–3–8–6–2–1. Множини DSU та E_T залишилися без змін, тому окремий рядок для цього кроку в таблиці 5.4 не наведено.

Ребра (6,7)=7, (7,8)=7, (2,4)=8 і (5,6)=9 не переглядаються, оскільки після кроку k = 7 дерево вже містить |V| – 1 = 7 ребер (усі вони утворили б цикл).

Мінімальне кістякове дерево побудовано: E_T = {(1,3)=1, (6,8)=1, (3,8)=2, (2,6)=3, (4,5)=3, (5,7)=5, (1,4)=6}.

Його вартість: $e^* = \sum e^*_i = 1 + 1 + 2 + 3 + 3 + 5 + 6 = 21$.

5. Програмна реалізація алгоритму Крускала

Програмну реалізацію алгоритму Крускала мовою C++ наведено на лістингу 5.5. Граф подано списком ребер; для перевірки циклів використано DSU (масив parent, функції find_set та union_set).

Лістинг 5.5 – Програмний код алгоритму Крускала

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
#define edge pair<int,int>
class Graph {
private:
    vector<pair<int, edge>> G; // граф (список ребер)
    vector<pair<int, edge>> T; // МКД
    int *parent;               // DSU
    int V;                     // кількість вершин
public:
    Graph(int V);
    void AddWeightedEdge(int u, int v, int w);
    int find_set(int i);
    void union_set(int u, int v);
    void kruskal();
    void print();
};
Graph::Graph(int V) {
    this->V = V;
    parent = new int[V + 1];
    for (int i = 0; i <= V; i++)  // кожна вершина - окрема множина
        parent[i] = i;
    G.clear();
    T.clear();
```

```
}
void Graph::AddWeightedEdge(int u, int v, int w) {
    G.push_back(make_pair(w, edge(u, v)));
}
int Graph::find_set(int i) {
    if (i == parent[i])
        return i;
    return find_set(parent[i]);
}
void Graph::union_set(int u, int v) {
    parent[u] = parent[v];
}
void Graph::kruskal() {
    int uRep, vRep;
    sort(G.begin(), G.end()); // сортування за неспаданням ваги
    for (size_t i = 0; i < G.size(); i++) {
        uRep = find_set(G[i].second.first);
        vRep = find_set(G[i].second.second);
        if (uRep != vRep) {           // ребро не утворює цикл
            T.push_back(G[i]);        // додати до МКД
            union_set(uRep, vRep);
        }
        if ((int)T.size() == V - 1) break;
    }
}
void Graph::print() {
    int total = 0;
    cout << "Edge : Weight" << endl;
    for (size_t i = 0; i < T.size(); i++) {
        cout << T[i].second.first << " - " << T[i].second.second << " :  "
             << T[i].first << endl;
        total += T[i].first;
    }
    cout << "Total weight: " << total << endl;
}
int main() {
    Graph g(8);   // варіант 22: вершини 1..8
    g.AddWeightedEdge(1, 3, 1);
    g.AddWeightedEdge(1, 2, 4);
    g.AddWeightedEdge(1, 4, 6);
    g.AddWeightedEdge(2, 4, 8);
    g.AddWeightedEdge(2, 6, 3);
```

```
    g.AddWeightedEdge(3, 8, 2);
    g.AddWeightedEdge(4, 5, 3);
    g.AddWeightedEdge(5, 6, 9);
    g.AddWeightedEdge(5, 7, 5);
    g.AddWeightedEdge(6, 7, 7);
    g.AddWeightedEdge(6, 8, 1);
    g.AddWeightedEdge(7, 8, 7);
    g.kruskal();
    g.print();
    return 0;
}
```

Результат виконання програми наведено на лістингу 5.6.

Лістинг 5.6 – Результат виконання програми

```
Edge : Weight
1 - 3 :  1
6 - 8 :  1
3 - 8 :  2
2 - 6 :  3
4 - 5 :  3
5 - 7 :  5
1 - 4 :  6
Total weight: 21
```

Порівняння ручної та автоматизованої побудови наведено в таблиці 5.5.

Таблиця 5.5 – Порівняння ручної та програмної побудови МКД (алгоритм Крускала)

Крок	**Ручна побудова (таблиця 5.4)**	**Програма (лістинг 5.6)**
1	(1,3) = 1	1 - 3 : 1
2	(6,8) = 1	6 - 8 : 1
3	(3,8) = 2	3 - 8 : 2
4	(2,6) = 3	2 - 6 : 3

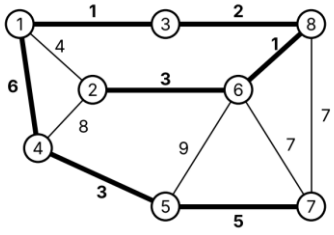
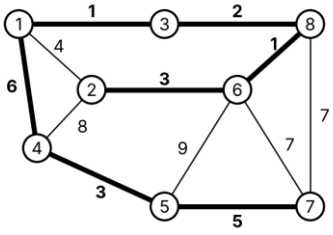
5	(4,5) = 3	4 - 5 : 3
6	(5,7) = 5	5 - 7 : 5
7	(1,4) = 6	1 - 4 : 6
Σ	21	Total weight: 21

Результати програми повністю збігаються з ручною побудовою: ті самі 7 ребер у тому самому порядку, загальна вага 21.

6. Порівняння результатів побудови МКД за двома алгоритмами

Результати побудови МКД за алгоритмами Пріма та Крускала зведено в таблиці 5.6.

Таблиця 5.6 – Порівняння МКД, побудованих алгоритмами Пріма та Крускала

Алгоритм Пріма		**Алгоритм Крускала**	
	(1,3) = 1 (3,8) = 2 (8,6) = 1 (6,2) = 3 (1,4) = 6 (4,5) = 3 (5,7) = 5 Σ = 21		(1,3) = 1 (6,8) = 1 (3,8) = 2 (2,6) = 3 (4,5) = 3 (5,7) = 5 (1,4) = 6 Σ = 21

Обидва алгоритми побудували одне й те саме мінімальне кістякове дерево з 7 ребер: {(1,3), (3,8), (6,8), (2,6), (1,4), (4,5), (5,7)} загальною вагою 21. Відрізняється лише послідовність додавання ребер. Алгоритм Пріма нарощує одне зв'язне дерево від вершини 1, тому додає ребра «по сусідству»: 1-3, 3-8, 8-6, 6-2, 1-4, 4-5, 5-7 – і змушений взяти ребро (1,4)=6 раніше за легші (4,5)=3 та (5,7)=5, бо вершини 4, 5, 7 ще не досяжні з дерева іншим шляхом. Алгоритм Крускала розглядає ребра

глобально за зростанням ваги, тому на проміжних етапах має ліс із кількох компонент ({1,2,3,6,8}, {4,5,7}), які наприкінці об'єднує ребром (1,4)=6.

Хоча у графі є ребра з однаковими вагами, МКД у цьому варіанті єдине: жодне з ребер-«двійників» не може бути замінене іншим без збільшення ваги (наприклад, ребро (1,2)=4 замикає цикл 1-3-8-6-2-1, у якому всі інші ребра легші). Тому розбіжностей у результатах немає.

За складністю: алгоритм Пріма на матриці суміжності (лістинг 5.2) має складність $O(|V|^3)$ у наведеній простій реалізації ($O(|V|^2)$ з масивом відстаней, $O(|E| \log |V|)$ з чергою з пріоритетами) і ефективніший для щільних графів. Алгоритм Крускала у наведеній реалізації (лістинг 5.5) має складність $O(|E| \log |E| + |E| \cdot |V|)$: сортування ребер виконується за $O(|E| \log |E|)$, а функція find_set без стискання шляхів може проходити ланцюжок довжиною до |V|. Якщо доповнити DSU стисканням шляхів та об'єднанням за рангом, операції FIND/UNION виконуються майже за константний час і загальна складність становить $O(|E| \log |E|)$. Алгоритм Крускала ефективніший для розріджених графів, до яких належить граф варіанта ($|E| = 12$ при максимально можливих 28).

Висновки: у ході виконання лабораторної роботи № 5 для зваженого неорієнтованого графа варіанта № 22 (8 вершин, 12 ребер) було побудовано матрицю та список суміжності, а також мінімальне кістякове дерево двома жадібними алгоритмами – Пріма та Крускала. Хід побудови показано графічно в таблицях 5.2 і 5.4. Обидва алгоритми реалізовано мовою C++; результати програм збіглися з ручною побудовою (таблиці 5.3 і 5.5). Отримане МКД складається з ребер (1,3), (3,8), (6,8), (2,6), (1,4), (4,5), (5,7) і має вагу 21. Алгоритми дали однакове дерево, але різний порядок додавання ребер: Прім розширює одне дерево від стартової вершини, а Крускал додає найлегші ребра по всьому графу, об'єднуючи компоненти за допомогою DSU. Таким чином, мету роботи – набуття практичних навичок проектування, реалізації, тестування та аналізу алгоритмів Пріма і Крускала – досягнуто.

Посилання на GitHub: https://github.com/TserkovniyAlex/ta-lab5-22.git